



PROYECTO

NeuroHealth Desktop

Sistema de Triage de Emergencias con

Windows Forms, Programación Orientada a Objetos y Persistencia en Archivos

1. Introducción	4
2. Contexto del sistema	4
3. Finalidad del proyecto	5
4. Objetivo general	5
5. Modelo general de la aplicación	5
6. Organización sugerida del proyecto	6
7. Responsabilidad de cada carpeta	7
Dominio	7
Enums	7
Forms	7
Interfaces	8
Repositorios	8
Servicios	8
8. Regla de separación de responsabilidades	9
9. Modelo de dominio	9
Clase Paciente	10
Clases derivadas de Paciente	10
Paciente de guardia	10
Paciente pediátrico	11
Clase SignosVitales	11
Clase Observacion	11
10. Aplicación de conceptos de Programación Orientada a Objetos	12
Clases y objetos	12
Asociación	12
Herencia	12
Polimorfismo	12
Interfaces	13
Genericidad	13
11. Funcionalidades obligatorias del sistema	14
Registro de pacientes	14
Pacientes en espera	15
Evaluación de pacientes mediante triaje	15
Reglas de triaje	15



Nivel Rojo	15
Nivel Amarillo	16
Nivel Verde	16
Pacientes admitidos	16
Registro de observaciones médicas	16
Consulta de observaciones	17
Estadísticas generales	17
Uso de tarea en segundo plano	17
12. Persistencia en archivos	18
Archivo de pacientes	18
Persistencia de la herencia	19
Archivo de observaciones	19
Repositorios de archivos	20
Carga inicial de datos	20
Archivos de prueba	20
13. Interfaz gráfica esperada	21
Formulario principal	21
Formulario de registro de pacientes	22
Formulario de observaciones	23
Formulario de estadísticas	23
Mensajes al usuario	23
Actualización de la interfaz	24
14. Validaciones y reglas de negocio	24
Validaciones del registro de pacientes	24
Rangos permitidos para signos vitales	25
Validaciones según tipo de paciente	25
Paciente de guardia	25
Paciente pediátrico	25
Validación de DNI repetido	25
15. Reglas para la evaluación de pacientes	26
Reglas para registrar observaciones	26
Reglas para consultar observaciones	26
Reglas para estadísticas	26
16. Tratamiento de errores	27
17. Esqueleto provisto por la cátedra	27
Uso esperado del esqueleto	28
18. Aspectos que deberán quedar implementados	28



Documentación dentro del código	29
Informe breve o bloque de explicación	29
19. Pruebas mínimas esperadas	29
20. Consideraciones sobre el uso de inteligencia artificial	30
21. Criterios de evaluación	30
22. Funcionamiento general de la aplicación	30
23. Aplicación de Programación Orientada a Objetos	31
Herencia y polimorfismo	31
Interfaces y repositorios	32
24. Persistencia en archivos	32
25. Interfaz gráfica y eventos	32
26. Separación de responsabilidades	33
27. Validaciones y manejo de errores	33
28. Comentarios y explicación del código	34
29. Presentación y prolijidad de la entrega	34
30. Trabajo en equipo y organización del desarrollo	34
Diagrama de clases UML	35
Nota aclaratoria sobre el ejecutable demostrativo	38
Fecha y Forma de Entrega	38

1. Introducción

En este proyecto se propone el desarrollo de una aplicación de escritorio en C# utilizando Windows Forms, orientada a la gestión inicial de pacientes en una guardia médica. La aplicación permitirá registrar pacientes, clasificarlos según su nivel de urgencia mediante un sistema de triaje, administrar observaciones médicas y consultar información general del sistema.

La propuesta surge como una evolución del proyecto trabajado previamente en consola, pero con un enfoque más cercano al desarrollo de software profesional. En esta nueva instancia, el objetivo no será solamente lograr que el programa funcione, sino también organizar el código aplicando conceptos de Programación Orientada a Objetos, separación de responsabilidades, uso de interfaces, persistencia en archivos y diseño de una interfaz gráfica simple.

El proyecto busca integrar distintos contenidos de la materia, especialmente clases, objetos, herencia, polimorfismo, interfaces, colecciones, manejo de archivos y formularios. Además, la estructura del sistema deberá quedar preparada para que, en una segunda etapa, la persistencia en archivos pueda ser reemplazada por una conexión a base de datos.

2. Contexto del sistema

La clínica NeuroHealth necesita una aplicación de escritorio que permita organizar el ingreso de pacientes a una guardia médica. Cuando un paciente llega, sus datos personales y signos vitales deben ser registrados en el sistema. Luego, el paciente permanece en una cola de espera hasta ser evaluado mediante un proceso de triaje.

El triaje consiste en clasificar al paciente según la gravedad de su estado. Para ello, el sistema deberá analizar datos como pulso, temperatura, saturación de oxígeno y nivel de dolor. A partir de esos valores, se asignará automáticamente un nivel de urgencia:

- Verde: paciente sin signos críticos.
- Amarillo: paciente con signos intermedios que requieren atención.
- Rojo: paciente crítico que requiere prioridad de atención.

Una vez evaluado, el paciente pasa a la lista de pacientes admitidos. A partir de ese momento, se podrán registrar observaciones médicas asociadas a su DNI.

3. Finalidad del proyecto

La finalidad de este proyecto es que los estudiantes puedan aplicar los contenidos trabajados en la materia en una situación concreta de desarrollo de software.

El proyecto permitirá trabajar especialmente los siguientes aspectos:

- Diseño de clases y objetos.
- Aplicación de herencia y polimorfismo.
- Uso de interfaces para separar responsabilidades.
- Organización del código en carpetas y archivos.
- Implementación de una aplicación de escritorio con Windows Forms.
- Uso de colecciones genéricas.
- Persistencia de datos mediante archivos de texto.
- Separación entre interfaz gráfica, lógica del sistema y almacenamiento de datos.
- Preparación del sistema para una futura conexión a base de datos.

La aplicación no debe resolverse colocando toda la lógica dentro de los formularios. Los formularios deberán encargarse principalmente de capturar datos, responder a eventos y mostrar información. La lógica del sistema deberá estar ubicada en clases de servicio, y el almacenamiento de datos deberá resolverse mediante repositorios.

4. Objetivo general

Desarrollar una aplicación de escritorio en C# con Windows Forms denominada NeuroHealth Desktop, que permita gestionar pacientes en una guardia médica, clasificarlos automáticamente mediante reglas de triaje, registrar observaciones médicas y persistir la información en archivos de texto, aplicando principios de Programación Orientada a Objetos y dejando preparada la estructura del sistema para una futura conexión a base de datos.

5. Modelo general de la aplicación

La aplicación deberá organizarse siguiendo una estructura simple por capas, con el objetivo de separar responsabilidades y facilitar futuras ampliaciones.

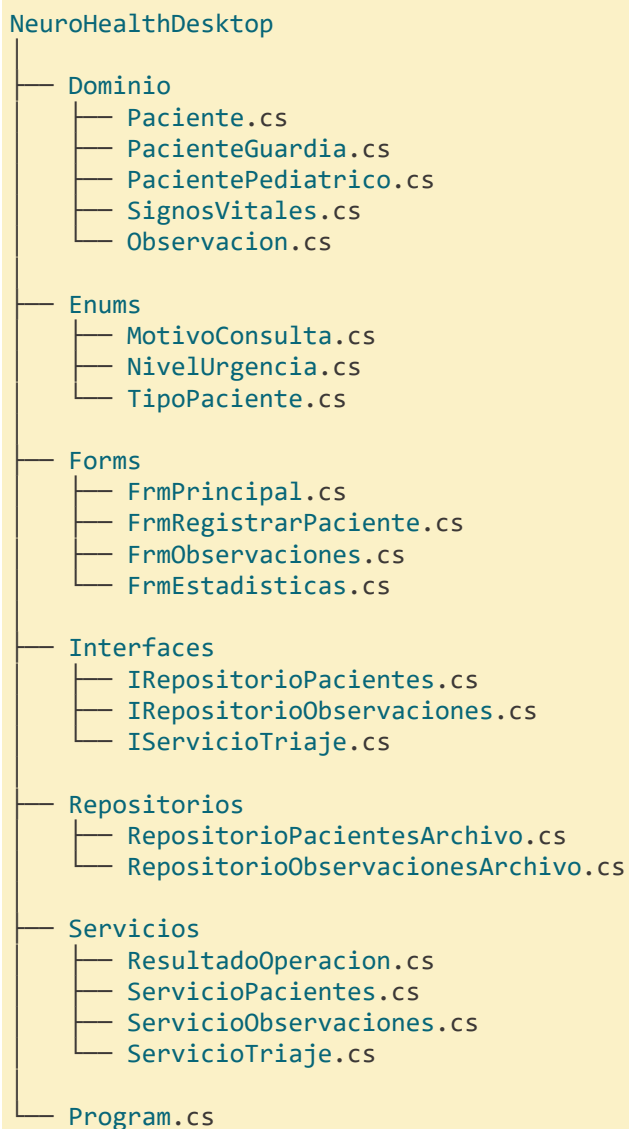
El sistema deberá distinguir claramente entre:

- Clases del dominio del problema.
- Enumeraciones.
- Interfaces.
- Servicios.
- Repositorios.
- Formularios de Windows Forms.

La separación del proyecto permitirá comprender mejor cómo se organiza una aplicación de escritorio y cómo se puede preparar un sistema para crecer sin modificar toda su estructura.

6. Organización sugerida del proyecto

El proyecto deberá organizarse en carpetas, respetando una estructura similar a la siguiente:



Esta organización busca que cada parte del sistema tenga una responsabilidad clara.

7. Responsabilidad de cada carpeta

Dominio

La carpeta **Dominio** deberá contener las clases principales del problema. Estas clases representan los elementos centrales del sistema, como pacientes, signos vitales y observaciones.

En esta carpeta deberán ubicarse, como mínimo:

- **Paciente**
- **PacienteGuardia**
- **PacientePediatico**
- **SignosVitales**
- **Observacion**

La clase **Paciente** deberá funcionar como clase base. A partir de ella deberán derivar dos tipos concretos de pacientes:

- **PacienteGuardia**
- **PacientePediatico**

Esto permitirá aplicar herencia y polimorfismo dentro del proyecto.

Enums

La carpeta **Enums** deberá contener tipos enumerados que representen conjuntos cerrados de valores utilizados por el sistema.

Como mínimo, se deberán utilizar:

- **MotivoConsulta**
- **NivelUrgencia**
- **TipoPaciente**

Estos valores no deberán cargarse como texto libre, sino seleccionarse desde opciones definidas por el sistema.

Forms

La carpeta **Forms** deberá contener los formularios de Windows Forms utilizados por la aplicación.

Como mínimo, la aplicación deberá contar con:

- **FrmPrincipal**: pantalla principal del sistema.
- **FrmRegistrarPaciente**: formulario para registrar pacientes.

- **FrmObservaciones**: formulario para registrar y consultar observaciones.
- **FrmEstadisticas**: formulario para mostrar indicadores generales.

Los formularios no deberán contener la lógica principal del sistema. Su función será capturar datos, responder a eventos, mostrar información y comunicarse con los servicios correspondientes.

Interfaces

La carpeta **Interfaces** deberá contener contratos que definan qué operaciones deben cumplir determinadas clases.

Como mínimo, se deberán incluir interfaces para:

- Repositorio de pacientes.
- Repositorio de observaciones.
- Servicio de triaje.

El uso de interfaces permitirá separar la lógica del sistema de la forma concreta en que se almacenan los datos. En esta primera etapa se utilizarán archivos, pero la estructura deberá permitir que en una segunda entrega se pueda reemplazar esa persistencia por una base de datos.

Repositorios

La carpeta **Repositorios** deberá contener las clases encargadas de guardar y recuperar información.

En esta primera entrega, la persistencia se realizará mediante archivos de texto. Por lo tanto, deberán implementarse repositorios que trabajen con archivos.

Como mínimo, se deberán incluir:

- **RepositorioPacientesArchivo**
- **RepositorioObservacionesArchivo**

Estos repositorios deberán encargarse de leer y escribir los datos en archivos, sin mezclar esa responsabilidad con los formularios ni con la lógica del triaje.

Servicios

La carpeta **Servicios** deberá contener la lógica principal del sistema.

Como mínimo, se deberán implementar:

- **ServicioPacientes**
- **ServicioObservaciones**

- ServicioTriage
- ResultadoOperacion

Los servicios deberán coordinar las operaciones del sistema, aplicar validaciones, invocar repositorios y devolver resultados comprensibles para la interfaz gráfica.

8. Regla de separación de responsabilidades

Una regla central del proyecto será la siguiente:

Los formularios no deben realizar cálculos principales, clasificaciones de triaje ni operaciones directas sobre archivos.

Los formularios deberán:

- Leer datos ingresados por el usuario.
- Validar aspectos básicos de pantalla.
- Crear objetos cuando corresponda.
- Llamar a los servicios.
- Mostrar mensajes y resultados.
- Actualizar grillas o listas visuales.

Los servicios deberán:

- Validar reglas del sistema.
- Registrar pacientes.
- Evaluar pacientes.
- Agregar observaciones.
- Calcular estadísticas.
- Coordinar el uso de repositorios.

Los repositorios deberán:

- Leer archivos.
- Escribir archivos.
- Buscar datos almacenados.
- Actualizar información persistida.

Esta separación permitirá que el proyecto sea más claro, mantenible y preparado para futuras modificaciones.

9. Modelo de dominio

El modelo de dominio representa los elementos principales del problema que se desea resolver. En este proyecto, el dominio está relacionado con la gestión de pacientes en una guardia médica.

Las clases principales del dominio serán:

- **Paciente**
- **PacienteGuardia**
- **PacientePediatico**
- **SignosVitales**
- **Observacion**

Estas clases deberán permitir representar pacientes, sus datos clínicos básicos, los distintos tipos de pacientes y las observaciones médicas asociadas.

Clase **Paciente**

La clase **Paciente** será la clase base del sistema. Representará los datos comunes que comparten todos los pacientes registrados en la guardia.

Como mínimo, deberá incluir:

- DNI.
- Apellido y nombre.
- Edad.
- Motivo de consulta.
- Signos vitales.
- Fecha y hora de ingreso.
- Nivel de urgencia.

La clase **Paciente** deberá definirse como clase base para otros tipos de pacientes. No se espera que el sistema trabaje con pacientes genéricos, sino con pacientes concretos derivados de esta clase.

Clases derivadas de **Paciente**

El sistema deberá trabajar con dos tipos de pacientes:

Paciente de guardia

La clase **PacienteGuardia** deberá representar a un paciente común de guardia. Además de los datos heredados desde **Paciente**, deberá indicar si el paciente requiere camilla.

Ejemplo de dato específico:

- Requiere camilla.

Paciente pediátrico

La clase **PacientePediátrico** deberá representar a un paciente menor de edad. Además de los datos heredados desde **Paciente**, deberá registrar el nombre del adulto responsable.

Ejemplo de dato específico:

- Adulto responsable.

Clase **SignosVitales**

La clase **SignosVitales** deberá agrupar los datos clínicos utilizados por el sistema para clasificar el nivel de urgencia del paciente.

Como mínimo, deberá incluir:

- Pulso.
- Temperatura.
- Presión arterial.
- Saturación de oxígeno.
- Nivel de dolor.

Esta clase se relaciona con **Paciente**, ya que cada paciente registrado deberá tener asociados sus signos vitales.

Clase **Observacion**

La clase **Observacion** deberá representar una observación médica asociada a un paciente.

Como mínimo, deberá incluir:

- Identificador de la observación.
- DNI del paciente asociado.
- Fecha y hora de la observación.
- Texto de la observación.

Las observaciones no deberán almacenarse dentro de la clase **Paciente**, sino administrarse mediante un repositorio independiente. Esto permitirá preparar el sistema para una futura base de datos, donde las observaciones podrán estar en una tabla separada.

10. Aplicación de conceptos de Programación Orientada a Objetos

El proyecto deberá evidenciar el uso de los siguientes conceptos:

Clases y objetos

Las clases representan los moldes o modelos del sistema. Los objetos son las instancias concretas que se crean durante la ejecución.

Ejemplo:

- `PacienteGuardia` es una clase.
- Un paciente con DNI 30111222 y nombre Ana Pérez es un objeto de esa clase.

Asociación

La asociación aparece cuando una clase se relaciona con otra.

En este proyecto, cada paciente tiene asociados signos vitales. Por lo tanto, existe una relación entre:

```
Paciente ---- SignosVitales
```

Además, cada observación estará asociada a un paciente mediante su DNI.

```
Paciente ---- Observacion
```

Herencia

La herencia deberá utilizarse para representar tipos específicos de pacientes.

La clase `Paciente` será la clase base, mientras que `PacienteGuardia` y `PacientePediatico` serán clases derivadas.

```
Paciente
├── PacienteGuardia
└── PacientePediatico
```

Esto permite evitar duplicación de código, ya que los datos comunes se definen una sola vez en la clase base.

Polimorfismo

El polimorfismo deberá evidenciarse mediante el uso de métodos redefinidos en las clases derivadas.

Por ejemplo, la clase **Paciente** podrá definir un método general para obtener la descripción del paciente, mientras que cada clase derivada podrá adaptar esa descripción según sus datos específicos.

De esta manera, una variable de tipo **Paciente** podrá referenciar objetos de tipo **PacienteGuardia** o **PacientePediatico**.

Interfaces

Las interfaces deberán utilizarse para definir contratos de comportamiento.

En este proyecto, se deberán utilizar interfaces para separar la lógica del sistema de la forma concreta en que se guardan los datos.

Por ejemplo:

```
IRepositorioPacientes  
IRepositorioObservaciones  
IServicioTriage
```

En esta primera entrega, las interfaces serán implementadas por clases que trabajan con archivos. En una segunda entrega, podrán ser implementadas por clases que trabajen con base de datos.

Genericidad

El proyecto deberá incluir el uso de genericidad mediante una clase de resultado de operación.

Esta clase permitirá devolver desde los servicios información sobre si una operación fue exitosa, un mensaje descriptivo y, cuando corresponda, un dato asociado.

Ejemplo conceptual:

```
ResultadoOperacion<Paciente>  
ResultadoOperacion<Observacion>
```

De esta manera, se podrá trabajar con una misma estructura para distintos tipos de datos.

11. Funcionalidades obligatorias del sistema

La aplicación deberá permitir realizar las siguientes operaciones principales:

1. Registrar pacientes.
2. Mostrar pacientes en espera.
3. Evaluar pacientes mediante triaje automático.
4. Mostrar pacientes admitidos.
5. Registrar observaciones médicas.
6. Consultar observaciones de un paciente.
7. Mostrar estadísticas generales.
8. Persistir la información en archivos.
9. Utilizar una tarea en segundo plano durante la evaluación del paciente.

Registro de pacientes

El sistema deberá permitir registrar nuevos pacientes desde un formulario de Windows Forms.

Para cada paciente se deberán cargar, como mínimo, los siguientes datos:

- DNI.
- Apellido y nombre.
- Edad.
- Motivo de consulta.
- Pulso.
- Temperatura.
- Presión arterial.
- Saturación de oxígeno.
- Nivel de dolor.
- Tipo de paciente.

El tipo de paciente deberá ser seleccionado entre las siguientes opciones:

- Paciente de guardia.
- Paciente pediátrico.

Si el paciente es de guardia, se deberá indicar si requiere camilla.

Si el paciente es pediátrico, se deberá indicar el adulto responsable.

Al registrarse, el paciente deberá quedar con nivel de urgencia **SinEvaluar** y deberá incorporarse a la cola de espera.

Pacientes en espera

El sistema deberá mostrar en la pantalla principal los pacientes que aún no fueron evaluados.

Estos pacientes representan la cola de espera del sistema. Un paciente permanecerá en esta cola hasta que se ejecute la evaluación de triaje.

La cola de espera deberá mostrar, como mínimo:

- DNI.
- Apellido y nombre.
- Edad.
- Motivo de consulta.
- Nivel.
- Tipo de paciente.

Evaluación de pacientes mediante triaje

El sistema deberá permitir evaluar al siguiente paciente en espera.

La evaluación deberá realizarse de manera automática, aplicando reglas de triaje sobre los signos vitales del paciente.

Una vez evaluado:

- el paciente deberá salir de la cola de espera;
- se le deberá asignar un nivel de urgencia;
- deberá incorporarse a la lista de pacientes admitidos;
- los datos deberán actualizarse en el archivo correspondiente.

Reglas de triaje

El sistema deberá clasificar a los pacientes en uno de los siguientes niveles:

- Verde
- Amarillo
- Rojo

Nivel Rojo

Un paciente será clasificado como **Rojo** si cumple al menos una de las siguientes condiciones:

- Saturación de oxígeno menor a 90.
- Pulso mayor a 120.
- Temperatura mayor o igual a 39.0.

- Dolor mayor o igual a 9.

Nivel Amarillo

Un paciente será clasificado como **Amarillo** si no fue clasificado como **Rojo** y cumple al menos una de las siguientes condiciones:

- Saturación de oxígeno entre 90 y 94.
- Pulso entre 100 y 120.
- Temperatura entre 38.0 y 38.9.
- Dolor entre 6 y 8.

Nivel Verde

Un paciente será clasificado como **Verde** si no cumple condiciones para ser clasificado como **Rojo** ni como **Amarillo**.

Pacientes admitidos

La aplicación deberá mostrar en la pantalla principal los pacientes que ya fueron evaluados.

La tabla de pacientes admitidos deberá mostrar, como mínimo:

- DNI.
- Apellido y nombre.
- Edad.
- Motivo de consulta.
- Tipo de paciente.
- Nivel de urgencia.

El nivel de urgencia deberá representarse visualmente mediante color:

- Rojo para pacientes críticos.
- Amarillo para pacientes de urgencia intermedia.
- Verde para pacientes sin signos críticos.

Esta representación visual permitirá identificar rápidamente la gravedad de cada paciente.

Registro de observaciones médicas

El sistema deberá permitir registrar observaciones médicas asociadas a un paciente.

Para registrar una observación se deberá ingresar:

- DNI del paciente.
- Texto de la observación.

Solo se podrán registrar observaciones para pacientes existentes y previamente evaluados.

No se deberán permitir observaciones para pacientes que se encuentren todavía en la cola de espera con nivel **SinEvalu**ar.

Consulta de observaciones

El sistema deberá permitir consultar las observaciones registradas para un paciente determinado, ingresando su DNI.

Las observaciones deberán mostrarse ordenadas desde la más reciente hacia la más antigua.

Cada observación deberá mostrar:

- Fecha y hora.
- Texto de la observación.

Estadísticas generales

El sistema deberá contar con un formulario de estadísticas que muestre información general sobre el estado de la guardia.

Como mínimo, deberá mostrar:

- Cantidad de pacientes en espera.
- Cantidad de pacientes admitidos.
- Cantidad de pacientes con nivel Verde.
- Cantidad de pacientes con nivel Amarillo.
- Cantidad de pacientes con nivel Rojo.
- Edad promedio de los pacientes admitidos.
- Porcentaje de pacientes críticos.

Las estadísticas deberán calcularse desde la lógica del sistema y no directamente desde el formulario.

Uso de tarea en segundo plano

La evaluación de pacientes deberá simular un proceso que tarda algunos segundos.

Para ello, se deberá utilizar una tarea en segundo plano mediante **async** y **await**, evitando que la interfaz gráfica quede bloqueada durante la evaluación.

Durante la evaluación, la aplicación deberá:

- deshabilitar temporalmente el botón de evaluación;



- mostrar una barra de progreso o indicador visual;
- informar al usuario que el paciente está siendo evaluado;
- actualizar la pantalla al finalizar el proceso.

Esta funcionalidad tiene como finalidad introducir de manera simple la idea de programación multihilo en aplicaciones de escritorio. No se pretende que los estudiantes desarrollen una solución avanzada de concurrencia ni que implementen mecanismos complejos de sincronización.

NOTA: A los fines de simplificar la implementación, la cátedra proveerá el método o fragmento de código necesario para realizar esta simulación. Los estudiantes deberán integrarlo correctamente al formulario principal, comprender su funcionamiento general y explicar brevemente por qué permite evitar que la interfaz gráfica quede bloqueada durante la evaluación.

12. Persistencia en archivos

En esta primera entrega, la aplicación deberá persistir la información utilizando archivos de texto.

La persistencia deberá estar implementada en clases específicas de repositorio. Los formularios no deberán leer ni escribir archivos directamente.

Como mínimo, el sistema deberá utilizar dos archivos:

- pacientes.txt
- observaciones.txt

El archivo **pacientes.txt** almacenará la información de los pacientes registrados.

El archivo **observaciones.txt** almacenará las observaciones médicas asociadas a los pacientes.

Archivo de pacientes

El archivo **pacientes.txt** deberá almacenar un paciente por línea.

Cada línea deberá contener los datos separados por el carácter |.

El formato sugerido será el siguiente:

```
TipoPaciente|Dni|NombreApellido|Edad|Motivo|Pulso|Temperatura|Presion|Saturacion|Dolor|FechaIngreso|Nivel|RequiereCamilla|AdultoResponsable
```

Ejemplo de paciente de guardia:

```
Guardia|30111222|Ana Perez|45|DolorToracico|125|38.7|130/80|89|9|2026-06-08
```



15:30:00|Rojo|True|

Ejemplo de paciente pediátrico:

Pediatrico|40111222|Juan Gomez|10|Fiebre|110|38.5|110/70|95|7|2026-06-08
15:35:00|Amarillo|Maria Gomez

El primer campo, **TipoPaciente**, permitirá reconstruir correctamente el objeto al leer el archivo. De esta manera, el sistema podrá identificar si debe crear un objeto de tipo **PacienteGuardia** o un objeto de tipo **PacientePediatico**.

Persistencia de la herencia

El sistema deberá guardar correctamente los datos específicos de cada tipo de paciente.

Para pacientes de guardia, se deberá guardar el dato:

- **RequiereCamilla**

Para pacientes pediátricos, se deberá guardar el dato:

- **AdultoResponsable**

Cuando un campo no corresponda al tipo de paciente, podrá quedar vacío.

Por ejemplo, un paciente de guardia no tendrá adulto responsable, por lo tanto el último campo podrá quedar vacío.

Del mismo modo, un paciente pediátrico no utilizará el campo **RequiereCamilla**, por lo tanto ese campo podrá quedar vacío.

Esta forma de almacenamiento permitirá trabajar la herencia en archivos y dejará preparada la estructura para que, en una segunda entrega, el sistema pueda guardar los mismos datos en una base de datos.

Archivo de observaciones

El archivo **observaciones.txt** deberá almacenar una observación por línea.

Cada línea deberá contener los datos separados por el carácter |.

El formato sugerido será el siguiente:

Id|DniPaciente|Fecha|Texto

Ejemplo:



1|30111222|2026-06-08 15:45:00|Paciente ingresa con dolor toracico persistente.

Otro ejemplo:

2|30111222|2026-06-08 15:55:00|Se administra oxigeno y se avisa al medico de guardia.

El campo **DniPaciente** permitirá asociar la observación con el paciente correspondiente.

Repositorios de archivos

El acceso a los archivos deberá realizarse mediante clases de repositorio.

Como mínimo, deberán existir:

RepositorioPacientesArchivo
RepositorioObservacionesArchivo

Estas clases deberán implementar las interfaces correspondientes:

IRepositorioPacientes
IRepositorioObservaciones

Los repositorios deberán encargarse de:

- crear los archivos si no existen;
- leer los datos desde archivos;
- convertir líneas de texto en objetos;
- convertir objetos en líneas de texto;
- agregar nuevos registros;
- actualizar registros existentes cuando corresponda.

Carga inicial de datos

Al iniciar la aplicación, el sistema deberá leer los datos existentes en los archivos.

Los pacientes con nivel **SinEvaluar** deberán aparecer en la cola de espera.

Los pacientes con nivel **Verde**, **Amarillo** o **Rojo** deberán aparecer en la lista de pacientes admitidos.

De esta manera, la aplicación deberá conservar su información aunque se cierre y se vuelva a abrir.

Archivos de prueba

La cátedra entregará archivos de prueba con pacientes y observaciones ya cargados.

Estos archivos tendrán como finalidad facilitar la verificación inicial del sistema y permitir que los estudiantes prueben la aplicación sin tener que cargar todos los datos manualmente.

Los estudiantes podrán utilizar estos archivos para comprobar:

- lectura correcta de pacientes;
- reconstrucción correcta de pacientes de guardia y pediátricos;
- carga de pacientes en espera;
- carga de pacientes admitidos;
- consulta de observaciones;
- cálculo de estadísticas;
- actualización de datos luego de evaluar pacientes.

13. Interfaz gráfica esperada

La aplicación deberá desarrollarse utilizando Windows Forms.

La interfaz gráfica deberá ser simple, clara y funcional. No se evaluará principalmente el diseño visual, sino que los formularios permitan operar correctamente el sistema y que estén conectados con la lógica correspondiente.

Como mínimo, la aplicación deberá contar con los siguientes formularios:

```
FrmPrincipal  
FrmRegistrarPaciente  
FrmObservaciones  
FrmEstadísticas
```

Formulario principal

El formulario principal será la pantalla central del sistema.

Desde este formulario se deberá poder:

- visualizar pacientes en espera;
- visualizar pacientes admitidos;
- registrar un nuevo paciente;
- evaluar el siguiente paciente;
- acceder al formulario de observaciones;
- acceder al formulario de estadísticas;
- actualizar la información mostrada;
- salir de la aplicación.

El formulario principal deberá mostrar dos tablas o grillas:



Pacientes en espera
Pacientes admitidos

La grilla de pacientes en espera deberá mostrar los pacientes cuyo nivel sea **SinEvaluar**.

La grilla de pacientes admitidos deberá mostrar los pacientes que ya fueron evaluados y tienen nivel **Verde**, **Amarillo** o **Rojo**.

En la grilla de pacientes admitidos, el nivel de urgencia deberá representarse visualmente mediante color.

Formulario de registro de pacientes

El formulario de registro de pacientes deberá permitir cargar los datos necesarios para crear un nuevo paciente.

Como mínimo, deberá permitir ingresar o seleccionar:

- DNI.
- Apellido y nombre.
- Edad.
- Motivo de consulta.
- Pulso.
- Temperatura.
- Presión arterial.
- Saturación de oxígeno.
- Nivel de dolor.
- Tipo de paciente.

El tipo de paciente deberá permitir seleccionar entre:

Guardia
Pediátrico

Si se selecciona **Guardia**, el formulario deberá permitir indicar si el paciente requiere camilla.

Si se selecciona **Pediátrico**, el formulario deberá permitir ingresar el nombre del adulto responsable.

El formulario deberá crear el objeto correspondiente según el tipo seleccionado:

- **PacienteGuardia**
- **PacientePediátrico**

Luego, deberá enviar ese objeto al servicio correspondiente para su registro.

Formulario de observaciones

El formulario de observaciones deberá permitir registrar y consultar observaciones médicas asociadas a pacientes.

Como mínimo, deberá permitir:

- ingresar el DNI de un paciente;
- buscar las observaciones registradas para ese paciente;
- ingresar una nueva observación;
- agregar la observación al sistema;
- mostrar las observaciones existentes.

Las observaciones deberán mostrarse desde la más reciente hacia la más antigua.

El formulario no deberá guardar observaciones directamente en archivos. Deberá comunicarse con el servicio de observaciones, y este con el repositorio correspondiente.

Formulario de estadísticas

El formulario de estadísticas deberá mostrar información general del sistema.

Como mínimo, deberá mostrar:

- cantidad de pacientes en espera;
- cantidad de pacientes admitidos;
- cantidad de pacientes con nivel Verde;
- cantidad de pacientes con nivel Amarillo;
- cantidad de pacientes con nivel Rojo;
- edad promedio de pacientes admitidos;
- porcentaje de pacientes críticos.

El formulario no deberá calcular directamente estos valores. Deberá solicitarlos al servicio correspondiente.

Mensajes al usuario

La aplicación deberá mostrar mensajes claros ante las operaciones principales.

Por ejemplo:

- paciente registrado correctamente;
- paciente evaluado correctamente;
- no hay pacientes en espera;
- observación registrada correctamente;

- no existe un paciente con ese DNI;
- solo se pueden registrar observaciones para pacientes evaluados;
- datos inválidos o incompletos.

Los mensajes deberán permitir que el usuario comprenda qué ocurrió y qué debe corregir en caso de error.

Actualización de la interfaz

Luego de cada operación importante, la interfaz deberá actualizar la información mostrada.

Por ejemplo:

- después de registrar un paciente, deberá actualizarse la cola de espera;
- después de evaluar un paciente, deberá actualizarse la cola de espera y la lista de admitidos;
- después de registrar una observación, deberá actualizarse el listado de observaciones;
- después de abrir el formulario de estadísticas, deberán mostrarse los valores actuales.

Esto permitirá que la aplicación refleje en pantalla el estado real del sistema.

14. Validaciones y reglas de negocio

La aplicación deberá incluir validaciones para evitar el registro de datos incorrectos, incompletos o inconsistentes.

Las validaciones podrán dividirse entre:

- validaciones básicas de formulario;
- validaciones propias de la lógica del sistema.

Los formularios podrán validar datos simples ingresados por el usuario, como campos vacíos o valores numéricos. Sin embargo, las reglas principales del sistema deberán estar ubicadas en las clases de servicio.

Validaciones del registro de pacientes

Al registrar un paciente, el sistema deberá verificar como mínimo:

- El DNI debe ser numérico.
- El DNI debe ser positivo.
- El DNI no debe estar repetido dentro del sistema.
- El apellido y nombre no debe estar vacío.

- La edad debe ser mayor a cero.
- El motivo de consulta debe seleccionarse desde las opciones disponibles.
- El tipo de paciente debe seleccionarse desde las opciones disponibles.
- La presión arterial no debe quedar vacía.
- Los signos vitales deben estar dentro de los rangos permitidos.

Rangos permitidos para signos vitales

Los signos vitales deberán respetar los siguientes rangos:

- Pulso: entre 30 y 200
- Temperatura: entre 34.0 y 42.0
- Saturación de oxígeno: entre 70 y 100
- Dolor: entre 0 y 10

La presión arterial podrá ingresarse como texto, por ejemplo:

- 120/80
- 130/90
- 110/70

Validaciones según tipo de paciente

El sistema deberá aplicar validaciones específicas según el tipo de paciente seleccionado.

Paciente de guardia

Si el paciente es de tipo **Guardia**, el sistema deberá permitir indicar si requiere camilla.

No será obligatorio cargar adulto responsable.

Paciente pediátrico

Si el paciente es de tipo **Pediatrico**, el sistema deberá solicitar obligatoriamente el nombre del adulto responsable.

No corresponderá cargar el dato de camilla.

Validación de DNI repetido

No se deberán permitir dos pacientes con el mismo DNI.

La validación de DNI repetido deberá realizarse consultando el repositorio de pacientes, de manera que se tenga en cuenta tanto la información cargada durante la ejecución actual como la información previamente guardada en archivo.

Si ya existe un paciente con ese DNI, el sistema deberá informar el error y no registrar el nuevo paciente.

15. Reglas para la evaluación de pacientes

Solo podrán evaluarse pacientes que estén en la cola de espera.

Al evaluar un paciente:

- deberá tomarse el siguiente paciente en espera;
- deberá aplicarse la clasificación de triaje;
- deberá asignarse el nivel de urgencia correspondiente;
- deberá quitarse de la cola de espera;
- deberá incorporarse a la lista de pacientes admitidos;
- deberá actualizarse el archivo de pacientes.

Si no hay pacientes en espera, el sistema deberá informar la situación al usuario.

Reglas para registrar observaciones

Solo se podrán registrar observaciones para pacientes existentes y previamente evaluados.

No se deberá permitir registrar observaciones en los siguientes casos:

- el DNI ingresado no corresponde a ningún paciente registrado;
- el paciente existe pero todavía tiene nivel **SinEvaluar**;
- el texto de la observación está vacío.

Cada observación deberá quedar asociada al paciente mediante su DNI.

Reglas para consultar observaciones

Al consultar observaciones, el usuario deberá ingresar el DNI del paciente.

El sistema deberá mostrar todas las observaciones asociadas a ese DNI.

Si el paciente no tiene observaciones registradas, deberá informarse esta situación.

Las observaciones deberán mostrarse desde la más reciente hacia la más antigua.

Reglas para estadísticas

Las estadísticas deberán calcularse considerando el estado actual del sistema.

Como mínimo:

- los pacientes en espera serán aquellos con nivel SinEvaluar;
- los pacientes admitidos serán aquellos con nivel Verde, Amarillo o Rojo;
- el promedio de edad deberá calcularse sobre pacientes admitidos;
- el porcentaje de pacientes críticos deberá calcularse sobre pacientes admitidos;
- se considerará paciente crítico a aquel que tenga nivel Rojo.

Si no existen pacientes admitidos, el sistema deberá evitar divisiones por cero y mostrar valores adecuados, por ejemplo 0 o 0.00 %.

16. Tratamiento de errores

La aplicación deberá manejar errores de manera clara para el usuario.

No se espera que la aplicación se cierre de forma abrupta ante un dato inválido o un problema de carga.

Cuando ocurra un error, el sistema deberá mostrar un mensaje comprensible indicando qué ocurrió.

Ejemplos:

- “El DNI debe ser numérico.”
- “Ya existe un paciente con ese DNI.”
- “Debe ingresar el adulto responsable del paciente pediátrico.”
- “No hay pacientes en espera.”
- “Solo se pueden registrar observaciones para pacientes evaluados.”

El tratamiento de errores deberá contribuir a que la aplicación sea usable y comprensible.

17. Esqueleto provisto por la cátedra

La cátedra podrá proporcionar un proyecto base o esqueleto inicial de la aplicación.

Este esqueleto tendrá como finalidad facilitar el inicio del desarrollo, evitar errores de configuración y permitir que los estudiantes se concentren en la comprensión, adaptación e implementación de las funcionalidades solicitadas.

El esqueleto podrá incluir:

- estructura inicial de carpetas;
- clases principales del dominio;

- enumeraciones;
- interfaces;
- formularios de Windows Forms;
- servicios parcialmente implementados;
- repositorios de archivos;
- archivos de prueba;
- fragmentos de código para la simulación de tarea en segundo plano.

El hecho de contar con un esqueleto no significa que el proyecto esté resuelto. Los estudiantes deberán comprender el código provisto, completarlo, adaptarlo, corregirlo y justificar las decisiones tomadas.

Uso esperado del esqueleto

Los estudiantes deberán utilizar el esqueleto como punto de partida para desarrollar la aplicación.

Se espera que puedan:

- identificar la responsabilidad de cada carpeta;
- comprender la relación entre formularios, servicios y repositorios;
- completar las funcionalidades pendientes;
- adaptar el código cuando sea necesario;
- corregir errores de integración;
- probar el funcionamiento completo de la aplicación;
- explicar las decisiones de diseño implementadas.

No se aceptará una entrega que consista únicamente en copiar archivos sin evidenciar comprensión del funcionamiento general del sistema.

18. Aspectos que deberán quedar implementados

La entrega deberá incluir, como mínimo:

- registro de pacientes de guardia y pacientes pediátricos;
- validación de datos obligatorios;
- cola de pacientes en espera;
- evaluación automática mediante reglas de triaje;
- visualización de pacientes admitidos;
- representación visual del nivel de urgencia;
- registro de observaciones médicas;
- consulta de observaciones por DNI;
- estadísticas generales;
- persistencia en archivos;
- lectura de datos al iniciar la aplicación;
- actualización de archivos luego de registrar o evaluar pacientes;

- uso de una tarea en segundo plano para simular la evaluación.

Documentación dentro del código

El código deberá incluir comentarios explicativos en las partes principales.

No se espera comentar cada línea, pero sí explicar las decisiones importantes.

Como mínimo, deberán estar comentados o explicados:

- la función de las clases principales;
- el motivo de uso de herencia entre pacientes;
- el uso de interfaces;
- la finalidad de los repositorios;
- la separación entre formularios, servicios y repositorios;
- el criterio utilizado para clasificar el triaje;
- el modo en que se guardan y recuperan datos desde archivos;
- el uso de la tarea en segundo plano durante la evaluación.

Los comentarios deberán demostrar comprensión. No deberán limitarse a repetir literalmente el nombre del método.

Informe breve o bloque de explicación

Además del código, el grupo deberá incluir una breve explicación del proyecto.

Esta explicación podrá entregarse como documento aparte o como un bloque de comentarios dentro del proyecto, según indique la cátedra.

La explicación deberá incluir:

- integrantes del grupo;
- descripción general del sistema;
- organización del proyecto;
- explicación de las clases principales;
- explicación de la herencia utilizada;
- explicación del uso de interfaces;
- explicación de la persistencia en archivos;
- explicación del flujo de evaluación de pacientes;
- dificultades encontradas y decisiones tomadas.

19. Pruebas mínimas esperadas

Antes de entregar, el grupo deberá probar como mínimo los siguientes casos:

1. Registrar un paciente de guardia.

2. Registrar un paciente pediátrico.
3. Intentar registrar un DNI repetido.
4. Evaluar un paciente en espera.
5. Verificar que el paciente evaluado pase a admitidos.
6. Verificar que el nivel de urgencia sea correcto.
7. Registrar una observación para un paciente evaluado.
8. Intentar registrar una observación para un paciente sin evaluar.
9. Consultar observaciones por DNI.
10. Cerrar la aplicación y volver a abrirla.
11. Verificar que los datos se conserven en los archivos.
12. Verificar que las estadísticas se actualicen correctamente.

Estas pruebas deberán realizarse antes de la entrega final para reducir errores de funcionamiento.

20. Consideraciones sobre el uso de inteligencia artificial

El uso de herramientas de inteligencia artificial no está prohibido, pero deberá realizarse de manera responsable.

Se espera que los estudiantes puedan comprender, explicar y modificar el código entregado.

No se considerará válida una entrega que evidencie copia automática de código sin comprensión, sin adaptación al proyecto o con funcionalidades que los integrantes no puedan explicar.

El uso de inteligencia artificial deberá ser entendido como una herramienta de asistencia, no como un reemplazo del trabajo de análisis, diseño, implementación y prueba del grupo.

21. Criterios de evaluación

La evaluación del proyecto tendrá en cuenta tanto el funcionamiento de la aplicación como la organización interna del código y la comprensión de los conceptos aplicados.

No se evaluará únicamente que la aplicación “ande”, sino también que esté correctamente diseñada, organizada y justificada.

22. Funcionamiento general de la aplicación

Se evaluará que la aplicación permita realizar correctamente las operaciones principales del sistema.

Se tendrá en cuenta:

- registro de pacientes;
- diferenciación entre paciente de guardia y paciente pediátrico;
- carga correcta de signos vitales;
- incorporación de pacientes a la cola de espera;
- evaluación automática mediante triaje;
- traslado de pacientes evaluados a la lista de admitidos;
- registro de observaciones;
- consulta de observaciones por DNI;
- visualización de estadísticas;
- conservación de datos al cerrar y volver a abrir la aplicación.

23. Aplicación de Programación Orientada a Objetos

Se evaluará el uso adecuado de los conceptos de Programación Orientada a Objetos trabajados en la materia.

Se tendrá en cuenta:

- correcta definición de clases;
- uso adecuado de atributos y propiedades;
- creación de objetos;
- uso de constructores;
- relación entre clases;
- aplicación de herencia;
- aplicación de polimorfismo;
- uso de interfaces;
- uso de colecciones genéricas;
- separación entre datos, lógica y presentación.

El proyecto deberá mostrar claramente que no se trata de una solución basada únicamente en formularios, sino de una aplicación organizada mediante clases.

Herencia y polimorfismo

Se evaluará especialmente la relación entre la clase base **Paciente** y sus clases derivadas.

El proyecto deberá evidenciar:

- una clase base que concentre datos comunes;
- clases derivadas que agreguen datos específicos;
- uso de variables o colecciones de tipo **Paciente** capaces de contener pacientes de distintos tipos;
- métodos que puedan comportarse de manera diferente según el tipo concreto de paciente.



No se considerará suficiente crear clases separadas sin relación entre sí. La herencia deberá estar correctamente aplicada y justificada.

Interfaces y repositorios

Se evaluará el uso de interfaces para separar el sistema de la forma concreta de almacenamiento.

El proyecto deberá incluir interfaces como contratos de comportamiento, por ejemplo:

```
IRepositorioPacientes  
IRepositorioObservaciones  
IServicioTriage
```

También se evaluará que los repositorios concretos implementen esas interfaces y que el resto del sistema trabaje, en lo posible, contra las abstracciones y no contra implementaciones concretas.

Este punto será importante para preparar el proyecto para una segunda etapa con base de datos.

24. Persistencia en archivos

Se evaluará que la aplicación guarde y recupere correctamente la información desde archivos de texto.

Se tendrá en cuenta:

- creación de archivos si no existen;
- lectura correcta de pacientes;
- lectura correcta de observaciones;
- escritura de nuevos pacientes;
- actualización del nivel de urgencia luego de evaluar;
- conservación de datos al cerrar la aplicación;
- reconstrucción correcta de pacientes de guardia y pacientes pediátricos desde el archivo;
- separación entre archivo de pacientes y archivo de observaciones.

Los formularios no deberán manipular archivos directamente.

25. Interfaz gráfica y eventos

Se evaluará que la aplicación utilice formularios de Windows Forms de manera adecuada.

Se tendrá en cuenta:

- claridad de los formularios;
- controles correctamente ubicados;
- eventos correctamente asociados;
- botones funcionales;
- grillas o listas actualizadas;
- mensajes claros al usuario;
- navegación simple entre formularios;
- representación visual del nivel de urgencia;
- actualización de la interfaz luego de cada operación.

La interfaz no necesita ser compleja ni visualmente avanzada, pero sí debe ser clara, usable y coherente con el funcionamiento del sistema.

26. Separación de responsabilidades

Se evaluará que el proyecto respete la separación entre:

- Formularios
- Servicios
- Repositorios
- Dominio

Los formularios deberán encargarse de la interacción con el usuario.

Los servicios deberán contener la lógica principal del sistema.

Los repositorios deberán encargarse del acceso a datos.

Las clases del dominio deberán representar los objetos centrales del problema.

Se descontará si toda la lógica del sistema se encuentra concentrada en los formularios.

27. Validaciones y manejo de errores

Se evaluará que el sistema controle datos inválidos y situaciones problemáticas.

Se tendrá en cuenta:

- validación de campos obligatorios;
- validación de DNI;
- control de DNI repetido;
- validación del adulto responsable en pacientes pediátricos;
- control de signos vitales fuera de rango;
- mensajes claros ante errores;
- manejo adecuado cuando no hay pacientes en espera;

- manejo adecuado cuando se intenta registrar una observación inválida.

La aplicación no deberá cerrarse inesperadamente por errores simples de carga de datos.

28. Comentarios y explicación del código

Se evaluará que el código incluya comentarios útiles y que los estudiantes puedan explicar lo desarrollado.

Se tendrá en cuenta:

- comentarios claros en clases y métodos importantes;
- explicación del uso de herencia;
- explicación del uso de interfaces;
- explicación de la persistencia;
- explicación del flujo de evaluación;
- explicación del uso de la tarea en segundo plano;
- coherencia entre el código y la explicación entregada.

No se valorarán comentarios automáticos, repetitivos o que no aporten comprensión.

29. Presentación y prolijidad de la entrega

Se evaluará la prolijidad general de la entrega.

Se tendrá en cuenta:

- proyecto correctamente organizado;
- nombres de clases, métodos y variables claros;
- ausencia de archivos innecesarios;
- solución que compile sin errores;
- instrucciones de ejecución si fueran necesarias;
- archivos de prueba correctamente ubicados;
- documentación breve incluida;
- entrega dentro de los plazos establecidos.

Una entrega prolija facilita la corrección y demuestra cuidado en el desarrollo del proyecto.

30. Trabajo en equipo y organización del desarrollo

El proyecto se realizará en los mismos equipos conformados para el Proyecto 1, salvo indicación expresa de la cátedra.

Dado que el trabajo se desarrolla en grupos y no se utilizarán herramientas de control de versiones como GitHub, cada equipo deberá organizarse de manera ordenada para evitar pérdidas de información, archivos duplicados o errores de integración.

Se recomienda que cada grupo:

- mantenga una única versión principal del proyecto;
- evite trabajar simultáneamente sobre copias distintas del mismo archivo;
- defina roles o responsabilidades de trabajo;
- designe un integrante responsable de consolidar los cambios;
- realice copias de seguridad periódicas del proyecto;
- pruebe la aplicación completa antes de la entrega;
- verifique que el proyecto compile correctamente en otra computadora;
- revise que todos los archivos necesarios estén incluidos en la entrega.

Los roles podrán organizarse, por ejemplo, de la siguiente manera:

- lógica del sistema y servicios;
- formularios y eventos;
- validaciones;
- repositorios y persistencia en archivos;
- pruebas, documentación e integración final.

La distribución de tareas no significa que cada integrante deba conocer únicamente “su parte”. Todos los integrantes del equipo deberán comprender el funcionamiento general de la aplicación, la estructura del proyecto y las principales decisiones de diseño adoptadas.

La falta de organización en el trabajo grupal suele derivar en errores frecuentes, tales como archivos desactualizados, formularios que no compilan, clases duplicadas, pérdida de cambios, nombres inconsistentes o funcionalidades que funcionan por separado pero fallan al integrarse.

Por tal motivo, se recomienda realizar reuniones breves de revisión, acordar qué integrante modifica cada archivo y probar el proyecto completo antes de la entrega final.

Diagrama de clases UML

Se incluye un diagrama de clases UML con la finalidad de ofrecer una representación visual de la estructura general del sistema NeuroHealth Desktop. Este diagrama permite observar las clases principales del proyecto, sus atributos, métodos y relaciones, incluyendo herencia, implementación de interfaces, asociaciones y dependencias entre componentes.

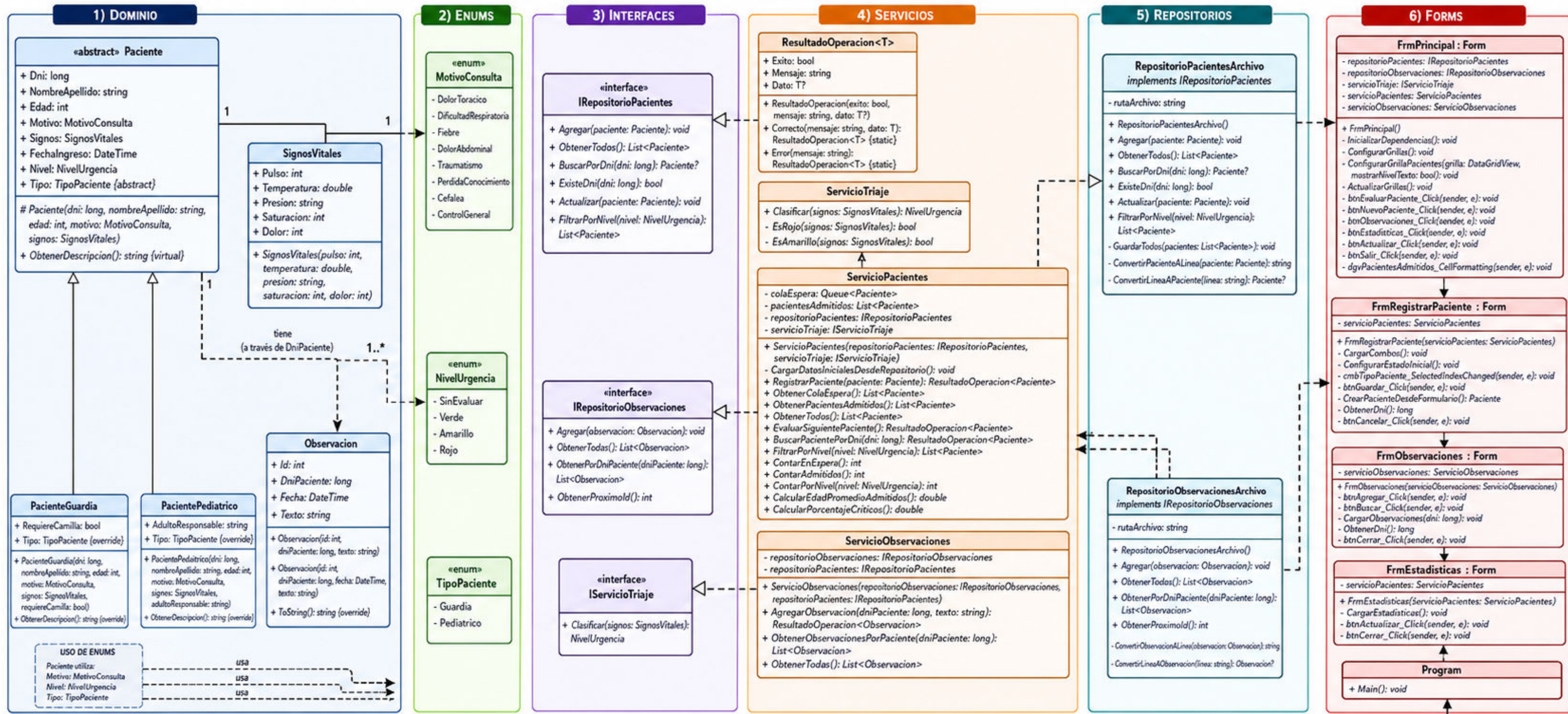


El objetivo del diagrama no es reemplazar la lectura y comprensión del código fuente, sino servir como una guía orientativa para interpretar cómo se organiza la aplicación. A través del diagrama, los estudiantes podrán identificar la separación entre dominio, servicios, repositorios, interfaces y formularios, así como comprender el modo en que se conectan las distintas partes del sistema.

Este recurso deberá utilizarse como apoyo para analizar el diseño de la solución, reconocer los conceptos de Programación Orientada a Objetos aplicados y facilitar la explicación del proyecto durante su desarrollo.

NeuroHealth Desktop - Diagrama de Clases UML

Proyecto C# / Windows Forms / Persistencia en archivos





Nota aclaratoria sobre el ejecutable demostrativo

A los fines demostrativos y orientativos, la cátedra podrá entregar un archivo ejecutable de la aplicación NeuroHealth Desktop.

Dicho ejecutable tendrá como finalidad permitir que los estudiantes observen el comportamiento general esperado del sistema, la navegación entre formularios, el flujo de registro de pacientes, la evaluación mediante triaje, la visualización de pacientes admitidos, el registro de observaciones y la consulta de estadísticas.

El ejecutable no reemplaza el desarrollo del proyecto ni constituye una solución cerrada que deba copiarse de manera automática. Su uso será únicamente de referencia para comprender el funcionamiento general esperado de la aplicación.

Los estudiantes deberán desarrollar, completar, adaptar y justificar su propio proyecto a partir del esqueleto provisto por la cátedra y de las consignas establecidas en el presente enunciado.

La existencia de un ejecutable demostrativo no exime a los estudiantes de comprender el código, explicar las decisiones de diseño, aplicar correctamente los conceptos de Programación Orientada a Objetos y respetar la separación entre formularios, servicios, repositorios y clases del dominio.

Fecha y Forma de Entrega

La entrega del proyecto se habilitará el día **jueves 25/06** y permanecerá abierta hasta el día **domingo 28/06 a las 23:59 hs.**

Cada grupo deberá entregar la **carpeta completa del proyecto comprimida en formato .zip**.

La entrega deberá incluir:

- todos los archivos fuente del proyecto;
- todos los archivos **.cs**;
- los formularios y sus archivos asociados;
- los archivos de recursos necesarios;
- el archivo **.csproj**;
- la solución, si corresponde;
- la carpeta de compilación correspondiente;
- el programa compilado dentro de la carpeta **Debug**.

Por “carpeta completa del proyecto” se entiende que no deberán entregar únicamente archivos sueltos ni capturas de pantalla, sino el proyecto completo, tal como fue trabajado en Visual Studio.



Antes de comprimir y entregar, el grupo deberá verificar que el proyecto compile correctamente y que la aplicación pueda ejecutarse desde Visual Studio.

Se recomienda realizar una copia final del proyecto, abrirla en otra ubicación de la computadora y comprobar que no falten archivos antes de subir la entrega.